Parallel Programming for High-Performance Computing

(PPAR, MU4IN903)

Sorbonne University

Golden Collision by the "MitM Attack"

By

Matthieu Bidault - (ID: 21401573)
Houssam Nasser - (ID: 21400407)

1 Parallelization of the Direct MITM Attack

To parallelize the given sequential algorithm, we used a sharding technique that divides the dictionary into equal parts among the MPI processes. Each process will have a pair $x \to f(x)$ (or $z \to g(z)$) in its shard if $f(x)$ (or $g(z)$) is equal to the process rank modulo the number of processes. This way, we can ensure that each process will have a set of pairs $x \to f(x)$ with no elements in common with other processes. Additionally, since we assume that f and g are uniformly distributed, we can also assume that the shards will be approximately equal in size.

For the implementation of the parallelized algorithm, we used two different methods. The first one uses MPI to comunicate buffers computed by each process using OpenMp threads. The second one uses OpenMP threads only with no communications. In both cases, to execute the attack, we will ask mpirun to run the program with one process per physical CPU (one process per socket). This way, each process will havec its memory allocated and the CPU will use its cores to run the OpenMP threads which are faster for shard memory access.

1.1 MPI + OpenMP Implementation

The idea behind this implementation is to divide the domain in equal parts amoung the MPI processes. Each process will compute different f(x) (and g(z)). Each process will use threads to compute its part, if the element computed is contained in the process' shard, it will be directly stored in the shard and if it is not, if will be stored in a buffer corresponding to the right process rank. Once all the elements are computed, the buffers are sent to the right process using MPI communications.

1.2 OpenMP Implementation

The idea behind this implementation is that the comunications are very slow and maybe we can avoid them completely. The idea is that each process will compute the whole domain and only store the elements that are in its shard. This way, we can avoid the MPI communications but also, in this way, we use way less data than the MPI + OpenMP implementation because we only store the shards and no buffers.

2 Solutions

The computed solutions for the given problems with various n, n being the size of the words, are shown in *Table 1.* The input of the problem can be found at `https://ppar.tme-crypto.fr/nasser.bidault/25` (replacing 25 with the real value).

3 Performance Analysis

For the performance analysis we ran the two algorithms on the Nantes and Rennes clusters of grid5000.

For Nantes, we used 100 nodes with each 1 socket and 18 cores per socket (that is 1800 cores in total) and 96GiB of RAM per nodes (that is 0.9TB in total) 6 hours at

n	Solution
25	(11e065a, 13bf7bf)
26	(151a80, 3831cf8)
27	(4e966f6, 2815842)
28	(22b8c0, f234406)
29	(b2292c0, 17ee4057)
30	(b1e00b9, 175e0f73)
31	(63fbc5aa, 581310d0)
32	(48068c9d, 27cc49fc)
33	(433ed0ae, 1c0d51f99)
34	(24123d4d7, d813bfbe)
35	(38606ddf0, 4db8079f9)
36	(d11c47308, 5e894e85a)
37	(3406de0, 1b4d8b904)
38	(19eff45498, 3a7b4940e3)
39	(695ee236ae, 5e0725fce0)

Table 1: *Computed solutions for various n*

night.

For Rennes, we used 29 nodes with each 2 sockets and 26 cores per socket (that is 1508 cores in total) and 384GiB of RAM per nodes (that is 10.9TB in total) for 6 hours at night.

3.1 Workload scaling

The workload scaling is the mesurement of the evolution of the execution time of the program with the number of processes being fixed and the input incressing in size. We will, in this section we compare the execution time evolution of our two implementations.

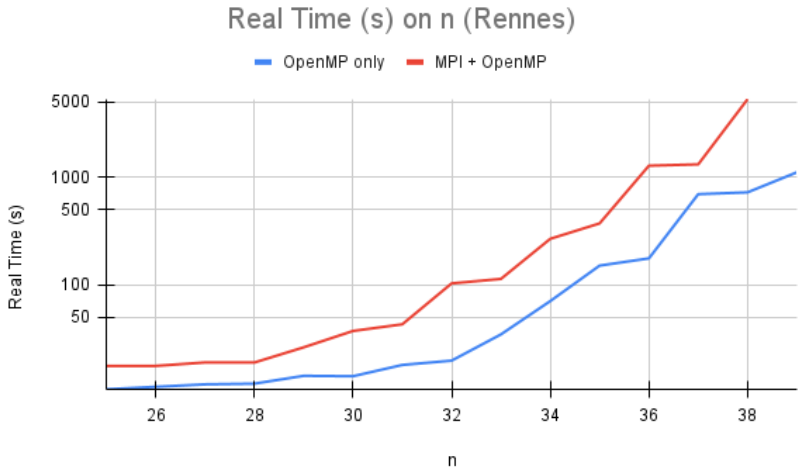


Figure 1: Workload scaling at Rennes

In those figures, *Figure 1* for the mesurement on the cluster at rennes and *Figure 2* for the mesurement on the cluster at nancy, we can see that the implementation using MPI

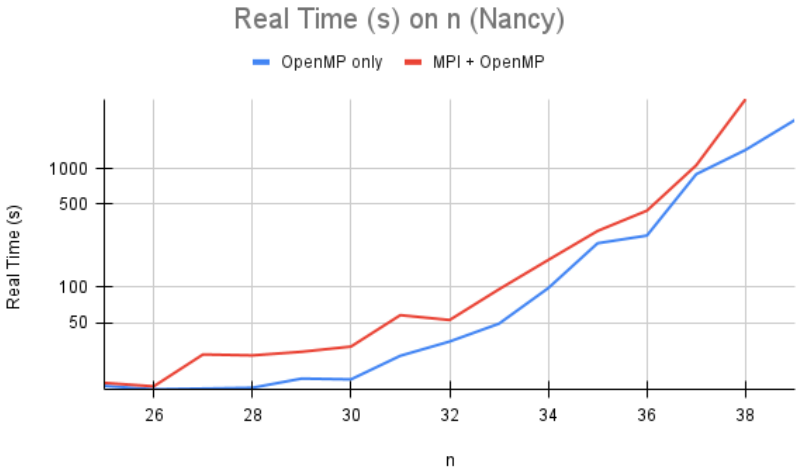


Figure 2: Workload scaling at Nancy

+ OpenMP is far slower than the implementation using OpenMP only. This is due to the fact taht compoting f(x) and g(x) is quite fast to compute and the MPI communications are very slow with a lot of nodes communicating.

Also we did not manage to run the MPI + OpenMP implementation with n ¿ 38 on the clusters because it was using to much memory for what was available. Similarly, we did not manage to run the OpenMP implementation with n ¿ 39.

For more details on the workload scaling, please refer to *Tables 2, 3, 4 and 5* in Annexes.

3.2 Strong scaling

The strong scaling is the mesurement of the evolution of the execution time of the program with the input size being fixed and the number of processes incressing. This way we can calculate the speedup (and efficiency) of the program. We must note that each process have multiple threads executing on dedicated physical cpu cores.

4 Conclusion

5 Annexes

Table 2:
Workload scaling at Rennes (OpenMP only)

n	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)
25	7.8	0.1	0.1	10.58
26	15.6	0.1	0.3	11.151
27	31.2	0.2	0.2	11.76
28	62.5	0.4	1.1	11.939
29	125	0.8	2.2	14.121
30	249.9	1.7	N/A	14.003
31	499.8	3.4	4	17.838
32	999.7	6.7	N/A	19.565
33	2000	13.5	N/A	34.355
34	4000	26.9	32.1	70.21
35	8000	53.9	64.2	150.302
36	16000	107.8	N/A	175.504
37	32000	215.5	N/A	696.251
38	64000	430.9	N/A	723.95
39	128000	862.5	N/A	1112.805

Table 3:
Workload scaling at Nancy (OpenMP only)

n	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)
25	4.6	0.1	0.1	14.785
26	9.2	0.2	0.3	13.853
27	18.5	0.4	0.6	14.053
28	37	0.7	1.1	14.265
29	74	1.4	2.3	17.052
30	147.9	2.9	N/A	16.786
31	295.8	5.8	6	26.516
32	591.7	11.4	N/A	34.968
33	1200	22.6	N/A	49.396
34	2400	45.4	N/A	98.507
35	4700	90.4	96.8	234.945
36	9500	179.9	78.4	272.642
37	18900	359.9	397.4	895.839
38	37900	719.8	N/A	1434.759
39	75700	1442.8	N/A	2560.983

Table 4:
Workload scaling at Rennes (MPI + OpenMP)

n	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)
25	7.8	6.1	0.7	17.459
26	15.6	5.8	1	17.47
27	31.2	6.8	0.7	18.818
28	62.5	7.4	0.1	18.783
29	125	9.8	5.2	26.026
30	249.9	13.8	11.7	36.977
31	499.8	22.4	8.6	42.659
32	999.7	36.9	53.4	102.609
33	2000	68.1	33	112.964
34	4000	132.3	119.1	267.064
35	8000	265.9	89.7	371.797
36	16000	535	715.9	1282.528
37	32000	1072.3	203.2	1319.91
38	64000	2138.5	3144.8	5339.673

Table 5:
Workload scaling at Nancy (MPI + OpenMP)

n	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)
25	4.5	0.7	0.3	15.684
26	9.1	0.8	0.5	14.66
27	18.1	12.6	1	27.215
28	36.2	13.2	0.4	26.721
29	72.5	14.5	0.5	28.661
30	145	17.1	1	31.699
31	289.9	23.1	21.6	58.286
32	579.8	34.8	4.5	53.056
33	1200	58.2	24	96.203
34	2300	105.6	49.8	169.981
35	4600	200.5	82.4	298.111
36	9300	390.7	29.7	441.882
37	18600	771.9	265.8	1064.773
38	37100	1546.3	2248.1	3833.082

Table 6:
Strong scaling at Rennes (MPI + OpenMP) (n = 31)

n processes	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)	Efficiency
1	29000	17,8	N/A	171,861	100,00%
2	14500	46,1	N/A	457,316	18,79%
3	9700	35,5	N/A	314,938	18,19%
4	7200	31,8	50,7	99,644	43,12%
5	5800	28,1	66,5	111,16	30,92%
6	4800	25,8	N/A	177,403	16,15%
7	4100	25,1	N/A	157,093	15,63%
8	3600	23	19,3	55,209	38,91%
9	3200	21,8	85,6	118,745	16,08%
10	2900	21,3	63,5	98,08	17,52%
11	2600	20,2	74,6	107,922	14,48%
12	2400	19,9	34	67,246	21,30%
13	2200	19,4	24,6	56,12	23,56%
14	2100	19,2	N/A	34,546	35,53%
15	1900	19,2	11,1	42,827	26,75%
16	1800	18,8	6,6	36,93	29,09%
17	1700	18,4	32,1	61,81	16,36%
18	1600	18,4	53	83,332	11,46%
19	1500	17,8	53,6	81,02	11,16%
20	1400	18	44,1	74,279	11,57%
21	1400	17,7	N/A	31,899	25,66%
22	1300	17,6	46,5	75,876	10,30%
23	1300	17,6	43	70,652	10,58%
24	1200	17,6	32,9	62,055	11,54%
25	1200	17,7	39,2	68,14	10,09%
26	1100	17,7	29	57,043	11,59%
27	1100	17,5	40,5	69,229	9,19%
28	1000	17,6	31,4	59,803	10,26%
29	999,7	17,5	21,5	49,953	11,86%
30	966,4	17,7	22,9	52,137	10,99%
31	935,2	17,5	20,7	48,276	11,48%
32	906	17,7	20,8	50,045	10,73%
33	878,5	17,6	36,2	64,99	8,01%
34	852,7	17,7	10,6	39,229	12,89%
35	828,3	17,7	8,2	36,961	13,29%
36	805,3	17,8	17,5	47,069	10,14%
37	783,5	17,8	26,8	55,293	8,40%
38	762,9	18	17,8	46,722	9,68%
39	743,4	18	15,4	44,069	10,00%
40	724,8	18,1	14,9	44,226	9,71%
41	707,1	18,1	22,9	51,673	8,11%
42	690,3	18,3	N/A	30,355	13,48%
43	674,2	18,3	17	46,731	8,55%

Continued on next page

Table 6 – continued from previous page

n processes	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)	Efficiency
44	658,9	18,5	15,8	45,489	8,59%
45	644,2	18,5	12,5	41,963	9,10%
46	630,2	18,8	15	43,255	8,64%
47	616,8	18,8	19,6	49,316	7,41%
48	604	19,1	11,5	41,407	8,65%
49	591,7	19	5,1	34,972	10,03%
50	579,8	19,2	29	59,082	5,82%
51	568,5	21	15,9	47,815	7,05%
52	557,5	21	10,6	42,002	7,87%
53	547	21,2	25,9	57,566	5,63%
54	536,9	21,6	29,8	62,407	5,10%
55	527,1	21,2	17,9	50,164	6,23%
56	517,7	22,1	26,6	59,315	5,17%
57	508,6	22,2	20,4	52,146	5,78%
58	499,8	22,1	8,3	41,013	7,22%

Table 7:
Strong scaling at Nancy (MPI + OpenMP) (n = 31)

n processes	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)	Efficiency
1	29000	26,6	N/A	176,006	100,00%
2	14500	57,5	N/A	490,494	17,94%
3	9700	43,9	N/A	338,085	17,35%
4	7200	36,4	55,5	106,686	41,24%
5	5800	31,1	69,1	114,625	30,71%
6	4800	28,1	10,3	52,411	55,97%
7	4100	24,8	7,8	44,393	56,64%
8	3600	23,4	18,8	54,645	40,26%
9	3200	21,6	86,7	120,084	16,29%
10	2900	20,4	63,8	97,249	18,10%
11	2600	19,2	75	107,484	14,89%
12	2400	18,8	34,4	66,036	22,21%
13	2200	17,8	25,2	54,564	24,81%
14	2100	17,4	3,6	32,628	38,53%
15	1900	16,6	11,7	40,48	28,99%
16	1800	16,8	7,3	35,181	31,27%
17	1700	16,4	32,4	60,18	17,20%
18	1600	16	52,9	80,388	12,16%
19	1500	15,7	53,9	80,848	11,46%
20	1400	15,5	44,4	71,811	12,25%
21	1400	15,3	2,3	29,222	28,68%
22	1300	15,2	46,8	72,844	10,98%
23	1300	15	43,4	68,618	11,15%

Continued on next page

Table 7 – continued from previous page

n processes	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)	Efficiency
24	1200	14,8	33	59,196	12,39%
25	1200	14,8	38,8	64,599	10,90%
26	1100	14,7	29	54,854	12,34%
27	1100	14,6	40,9	66,791	9,76%
28	1000	14,6	31,3	56,912	11,04%
29	999,7	14,4	21,5	47,176	12,86%
30	966,4	14,5	22,9	48,724	12,04%
31	935,2	14,5	20,6	44,958	12,63%
32	906	14,4	20,7	46,287	11,88%
33	878,5	14,5	36,1	61,672	8,65%
34	852,7	14,4	10,7	36,32	14,25%
35	828,3	14,5	8,2	33,629	14,95%
36	805,3	14,4	17,3	42,607	11,47%
37	783,5	14,4	26,7	51,967	9,15%
38	762,9	14,5	17,5	43,057	10,76%
39	743,4	14,5	15,3	40,938	11,02%
40	724,8	14,5	14,7	40,127	10,97%
41	707,1	14,5	22,7	47,887	8,96%
42	690,3	14,6	N/A	26	16,12%
43	674,2	14,7	16,9	42,573	9,61%
44	658,9	14,7	15,6	41,246	9,70%
45	644,2	14,8	12,3	37,785	10,35%
46	630,2	14,9	14,6	39,375	9,72%
47	616,8	14,9	19,2	45,456	8,24%
48	604	15	11,2	37,461	9,79%
49	591,7	15	4,9	30,934	11,61%
50	579,8	15,2	28	54,004	6,52%
51	568,5	15,3	15,4	41,47	8,32%
52	557,5	15,4	10	36,121	9,37%
53	547	15,5	24,5	50,62	6,56%
54	536,9	15,6	28	54,508	5,98%
55	527,1	15,6	16,8	43,233	7,40%
56	517,7	15,8	24,6	51,191	6,14%
57	508,6	15,9	18,7	45,094	6,85%
58	499,8	15,9	7,6	34,321	8,84%
59	491,4	16,1	26,9	54,008	5,52%
60	483,2	16,3	8,1	35,307	8,31%
61	475,3	16,4	8	34,825	8,29%
62	467,6	16,5	20,5	46,417	6,12%
63	460,2	16,6	14,9	42,561	6,56%
64	453	16,6	7,4	34,764	7,91%
65	446	16,8	7,2	34,92	7,75%
66	439,3	17	25,6	55,237	4,83%
67	432,7	17,1	12,6	42,652	6,16%

Continued on next page

Table 7 – continued from previous page

n processes	Dictionary Size (MB)	Fill Time (s)	Probe Time (s)	Real Time (s)	Efficiency
68	426,3	17,2	3,8	33,704	7,68%
69	420,2	17,4	16,4	46,679	5,46%
70	414,2	17,5	3	32,956	7,63%
71	408,3	17,7	8,5	39,028	6,35%
72	402,7	17,8	6,3	36,751	6,65%
73	397,1	18	11,1	41,993	5,74%
74	391,8	18,2	21,8	52,717	4,51%
75	386,5	18,3	15,7	46,734	5,02%
76	381,5	18,5	18,5	49,625	4,67%
77	376,5	18,6	7	38,323	5,96%
78	371,7	18,7	5,6	36,806	6,13%
79	367	19	17,5	49,191	4,53%
80	362,4	19,2	5,4	37,608	5,85%
81	357,9	19,4	15,8	47,948	4,53%
82	353,5	19,6	20,1	52,482	4,09%
83	349,3	19,8	17,9	50,53	4,20%
84	345,1	19,9	21,7	54,169	3,87%
85	341,1	20,1	21,4	54,245	3,82%
86	337,1	20,2	17,9	50,689	4,04%
87	333,2	20,5	12	45,215	4,47%
88	329,4	20,6	17,3	50,729	3,94%
89	325,7	20,8	3,9	37,263	5,31%
90	322,1	21	4,6	38,308	5,10%
91	318,6	21,2	14,3	48,076	4,02%
92	315,1	21,4	5,5	39,527	4,84%
93	311,7	21,6	11,7	46,07	4,11%
94	308,4	21,8	7,3	41,636	4,50%
95	305,2	22,1	13,7	48,56	3,82%
96	302	22,2	4,3	39,259	4,67%
97	298,9	22,4	9,7	44,939	4,04%
98	295,8	22,7	1,9	37,292	4,82%
99	292,8	22,9	22,2	57,689	3,08%
100	289,9	23,1	21,5	57,266	3,07%